TIP102 | Intermediate Technical Interview Prep

Intermediate Technical Interview Prep Fall 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: **134071**

Need help? Post on our **class slack channel** or email us at **support@codepath.org**

Getting Started

Learning with AI ✨

▶ Course Info

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

Week 11

Week 12

Week 5: Session 2

Linked Lists II

In this session, students will delve into the intricate world of linked lists, focusing on both singly and doubly linked list structures. They will engage in a series of hands-on problems that challenge them to perform operations such as insertion, deletion, copying, and value manipulation within linked lists.

> You can find session recordings and more on the resources tab. Session slide decks are available on the overview tab.

📈 Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.

👷 Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
    - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like VSCode, PyCharm, Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

▶ **Note on Expectations**

🔎 Problem Solving Approach

We will approach problems using the six steps in the UMPIRE approach.

UMPIRE: Understand, Match, Plan, Implement, Review, Evaluate.

We'll apply these six steps to the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem
- **Match** identifies common approaches you've seen/used before
- **Plan** a solution step-by-step, and
- **Implement** the solution
- **Review** your solution
- **Evaluate** your solution's time and space complexity and think critically about the advantages and disadvantages of your chosen approach.

Problem Set Version 1

▼ Problem 1: Mutual Friends

In the `Villager` class below, each villager has a `friends` attribute, which is a list of other villagers they are friends with.

Write a method `get_mutuals()` that takes one parameter, a `Villager` instance `new_contact`, and returns a list with the `name` of all friends the current villager and `new_contact` have in common.

```
class Villager:
    def __init__(self, name, species, catchphrase):
        self.name = name
        self.species = species
        self.catchphrase = catchphrase
        self.friends = []

    def get_mutuals(self, new_contact):
        pass
```

Example Usage:

```
bob = Villager("Bob", "Cat", "pthhhpth")
marshal = Villager("Marshal", "Squirrel", "sulky")
ankha = Villager("Ankha", "Cat", "me meow")
fauna = Villager("Fauna", "Deer", "dearie")
raymond = Villager("Raymond", "Cat", "crisp")
stitches = Villager("Stitches", "Cub", "stuffin")

bob.friends = [stitches, raymond, fauna]
marshal.friends = [raymond, ankha, fauna]
print(bob.get_mutuals(marshal))

ankha.friends = [marshal]
print(bob.get_mutuals(ankha))
```

Example Output:

```
['Raymond', 'Fauna']
[]
```

▶ ✨ **AI Hint: Class Methods**

Close Section

▼ Problem 2: Linked Up

A **linked list** is a data structure that, similar to a normal list or array, allows us to store pieces of data sequentially. The key difference is how the elements are stored in memory.

In a normal list, elements are stored in adjacent memory locations. If we know the location of the first element, we can easily access any other element in the list.

In a linked list, individual elements, called **nodes**, are not stored in sequential memory locations. Instead, each node stores a reference or pointer to the next node in the list, allowing us to traverse the list.

Connect the provided node instances below to create the linked list `kk_slider -> harriet -> saharah -> isabelle`.

A function `print_linked_list()` which accepts the **head**, or first element, of a linked list and prints the values of the list has also been provided for testing purposes.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
```

```
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

kk_slider = Node("K.K. Slider")
harriet = Node("Harriet")
saharah = Node("Saharah")
isabelle = Node("Isabelle")

# Add code here to link the above nodes
```

Example Usage:

```
print_linked_list(kk_slider)
```

Example Output:

```
K.K. Slider -> Harriet -> Saharah -> Isabelle
```

▶ ✨ **AI Hint: Linked Lists**

Close Section

▼ Problem 3: Daily Tasks

Imagine a linked list used as a daily task list where each node represents a task. Write a function `add_task()` that takes in the `head` of a linked list and adds a new node to the front of the task list.

The function should insert a new `Node` object with the value `task` as the new `head` of the linked list and return the new node.

Note: The "head" of a linked list is the first element in the linked list. It is equivalent to `lst[0]` of a normal list.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def add_first(head, task):
    pass
```

Example Usage:

```
task_1 = Node("shake tree")
task_2 = Node("dig fossils")
task_3 = Node("catch bugs")
task_1.next = task_2
task_2.next = task_3

# Linked List: shake tree -> dig fossils -> catch bugs
print_linked_list(add_first(task_1, "check turnip prices"))
```

Example Output:

```
check turnip prices -> shake tree -> dig fossils -> catch bugs
```

Close Section

▼ Problem 4: Halve List

Write a function `halve_list()` that accepts the `head` of a linked list whose values are integers and divides each value by two. Return the `head` of the modified list.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def halve_list(head):
    pass
```

Example Usage:

```
node_one = Node(5)
node_two = Node(6)
node_three = Node(7)
node_one.next = node_two
node_two.next = node_three

# Input List: 5 -> 6 -> 7
print_linked_list(halve_list(node_one))
```

Example Output:

```
2.5  -> 3 -> 3.5
```

▶ 💡 **Hint: Linked List Traversal**

Close Section

▼ Problem 5: Remove Last

Write a function `delete_tail()` that accepts the `head` of a linked list and removes the last node in the list. Return the `head` of the modified list.

Note: The "tail" of a list is the last element in the linked list. It is equivalent to `lst[-1]` in a normal list.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def delete_tail(head):
    pass
```

Example Usage:

```
butterfly = Node("Common Butterfly")
ladybug = Node("Ladybug")
beetle = Node("Scarab Beetle")
butterfly.next = ladybug
ladybug.next = beetle

# Input List: butterfly -> ladybug -> beetle
print_linked_list(delete_tail(butterfly))
```

Example Output:

```
Common Butterfly -> Ladybug
```

Close Section

▼ Problem 6: Find Minimum in Linked List

Write a function `find_min()` that takes in the `head` of a linked list and returns the minimum value in the linked list. You can assume the linked list will contain only numeric values.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def find_min(head):
    pass
```

Example Usage:

```
head1 = Node(5, Node(6, Node(7, Node(8))))
head2 = Node(8, Node(5, Node(6, Node(7))))

# Linked List: 5 -> 6 -> 7 -> 8
print(find_min(head1))

# Linked List: 8 -> 5 -> 6 -> 7
print(find_min(head2))
```

Expected Output:

```
5
5
```

▶ 💡 **Hint: Nested Constructors**

Close Section

▼ Problem 7: Remove From Inventory

Imagine a linked list used to store a player's inventory. Write a function `delete_item()` that takes in the `head` of a linked list and a value `item` as parameters.

The function should remove the first node it finds in the linked list with the value `item` and return the `head` of the modified list. If no node can be found with the value `item`, return the list unchanged.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def delete_item(head, item):
    pass
```

Example Usage:

```
slingshot = Node("Slingshot")
peaches = Node("Peaches")
beetle = Node("Scarab Beetle")
slingshot.next = peaches
peaches.next = beetle

# Linked List: slingshot -> peaches -> beetle
print_linked_list(delete_item(slingshot, "Peaches"))

# Linked List: slingshot -> beetle
print_linked_list(delete_item(slingshot, "Triceratops Torso"))
```

Example Output:

```
Slingshot -> Scarab Beetle
Slingshot -> Scarab Beetle
```

Close Section

▼ Problem 8: Move Tail to Front of Linked List

Write a function `tail_to_head()` that takes in the `head` of a linked list as a parameter and moves the tail of the linked list to the front.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# For testing
def print_linked_list(head):
    current = head
    while current:
```

```
        print(current.value, end=" -> " if current.next else "\n")
        current = current.next

def tail_to_head(head):
    pass
```

Example Usage:

```
daisy = Node("Daisy")
mario = Node("Mario")
toad = Node("Toad")
peach = Node("Peach")
daisy.next = mario
mario.next = toad
toad.next = peach

# Linked List: Daisy -> Mario -> Toad -> Peach
print_linked_list(tail_to_head(daisy))
```

Example Output:

```
Peach -> Daisy -> Mario -> Toad
```

Close Section

▼ Problem 9: Create Double Links

One of the drawbacks of a linked list is that it's difficult to go backwards because each `Node` only knows about the `Node` in front of it. (E.g., `A -> B -> C`)

A **doubly linked list** solves this problem! Instead of just having a `next` attribute, a doubly linked list also has a `prev` attribute that points to the `Node` before it. (E.g., `A <-> B <-> C`)

Update the `Node` constructor below so that the code creates a doubly linked list with `head <-> tail`.

```
class Node:
    def __init__(self, value, next=None, prev=None):
        self.value = value
        self.next = next
        self.prev = prev

head = Node("Isabelle")
tail = Node("K.K. Slider")

head.next = tail
tail.prev = head
```

Example Usage:

```
print(head.value, "<->", head.next.value)
print(tail.prev.value, "<->", tail.value)
```

Example Output:

```
Isabelle <-> K.K. Slider
Isabelle <-> K.K. Slider
```

Close Section

▼ Problem 10: Print Backwards

Write a function `print_reverse()` that takes in the `tail` of a doubly linked list as a parameter.

It should print out the values of the linked list in reverse order, each separated by a space.

```
class Node:
    def __init__(self, value, next=None, prev=None):
        self.value = value
        self.next = next
        self.prev = prev

def print_reverse(tail):
    pass
```

Example Usage:

```
isabelle = Node("Isabelle")
kk_slider = Node("K.K. Slider")
saharah = Node("Saharah")
isabelle.next = kk_slider
kk_slider.next = saharah
saharah.prev = kk_slider
kk_slider.prev = isabelle

# Linked List: Isabelle <-> K.K. Slider <-> Saharah
print_reverse(saharah)
```

Example Output:

```
Saharah K.K. Slider Isabelle
```

Close Section

Problem Set Version 2

▶ Problem 1: Calculate Tournament Placement

- Problem 2: Update Linked List Sequence
- Problem 3: Insert Node as Second Element
- Problem 4: Increment Linked List Node Values
- Problem 5: Copy Linked List
- Problem 6: Making the Cut
- Problem 7: Remove Racer
- Problem 8: Array to Linked List
- Problem 9: Convert Singly Linked List to Doubly Linked List
- Problem 10: Find Length of Doubly Linked List from Any Node